

Customer Churn Risk Prediction & MLOps Pipeline

Comprehensive Technical Report, Architectural Audit, CI/CD & Note Inferences

Project Domain: Multi-Class Customer Churn Risk (0-4)

ML Models Evaluated: XGBoost, LightGBM, CatBoost

MLOps & CI/CD: GitHub Actions, Docker Hub, MLflow, FastAPI

Cloud Platform: AWS ECS Fargate, ALB, CloudWatch

1. Executive Summary

This report provides a complete architectural and analytical breakdown of the Customer Churn Risk Prediction project. The objective of this system is to classify customer churn risk across 5 distinct risk categories (0 to 4), enabling the business to proactively engage drifting or unhappy customers while avoiding wasteful incentive spend on loyal segments.

The project implements a full production MLOps pipeline comprising data ingestion, custom data cleaning and MICE numerical imputation, hyperparameter optimization using Optuna cross-validation, experiment tracking via MLflow, automated CI/CD container build/push using GitHub Actions, web API & UI serving via FastAPI and Gradio, and container orchestration on AWS ECS Fargate behind an Application Load Balancer (ALB).

KEY HIGHLIGHTS & SUMMARY FINDINGS:

- **Champion Model:** Optuna-tuned **XGBoost Classifier** achieved **78.27% Test Accuracy** and **0.7689 Macro-F1** (0.7777 Weighted-F1), outperforming LightGBM (0.7585 Macro-F1) and CatBoost (0.7642 Macro-F1).
- **Empirical Objective Validation:** Directly comparing Macro F1 tuning vs. Recall tuning proved that **Macro F1 tuning wins on BOTH Macro-F1 (0.769 vs 0.764) AND Macro-Recall (0.775 vs 0.771)**. Recall-only search got trapped in shallow optima by over-saturating Class 4 recall (~0.99) while degrading Class 3 recall (0.547 -> 0.523).
- **Automated CI/CD Pipeline:** GitHub Actions workflow (.github/workflows/ci.yml) automatically builds modern multi-platform Docker images via Docker Buildx upon every push to main and pushes image csagnik1302/ml_img:latest to Docker Hub.
- **Cloud Deployment Architecture:** Containerized service deployed on AWS ECS Fargate (0.5 vCPU, 1 GB RAM) registered with an HTTP ALB target group, decoupled from ephemeral task IP changes and health-checked via GET /.

2. Business Context & Customer Segmentation Rubric

To interpret model predictions effectively, the business maps customer behaviors across 5 ordinal score levels (1 to 5 in raw business terms, mapped to zero-indexed classes 0 to 4 in the machine learning code):

Raw Score / ML Class	Customer Segment	Behavioral Profile & Financial Metrics	Business Action Strategy
Score 1 & 2 (Class 0 & 1)	Happy & Loyal Customers	Positive feedback only ('Products in Stock', 'Quality Customer Care', 'Reasonable Price'). Median transaction ~\$52k (2x other groups). Wallet points 750-800; logins every ~10 days. Exclusively Gold, Silver, Platinum, or Premium members.	Maintain satisfaction; avoid unnecessary promotional spend or spam.
Score 3 (Class 2)	Middling Customers	Negative feedback present; mid-tier membership. Median transaction value ~\$26k, wallet points ~750. Neutral/borderline engagement.	Targeted nurturing & feedback resolution.
Score 4 (Class 3)	Unhappy & Drifting	Negative feedback prevalent. Skews to Basic/No Membership, or lower Silver/Gold tiers. Wallet points ~\$660. Highest risk of sliding to active churn.	Proactive intervention; service quality remediation.
Score 5 (Class 4)	High Risk (Leaving)	Negative feedback dominant; 63-64% of all Basic and No Membership customers land here. Lowest wallet points (~600).	Aggressive win-back campaigns or high-impact retention offers.

3. End-to-End Data Pipeline & Feature Engineering

The automated data pipeline implemented in src/data/preprocess.py processes raw customer records through strict data hygiene steps:

- **Stratified Partitioning:** Applies an 80/20 train/test split stratified on churn_risk_score to preserve target distribution across partitions.
- **Column Drop & Renaming:** Removes non-predictive identifiers (customer_id, Name, security_no, referral_id, last_visit_time). Renames avg_time_spent to avg_time_spent_in_seconds.

- **Invalid/Placeholder Replacement:** Replaces sentinel value -999 in days_since_last_login with NaN. Replaces negative values in numeric fields and string 'Error' in avg_frequency_login_days with NaN.
- **Date Transformation:** Parses joining_date into datetime and computes elapsed days days_since_joined = ('2021-04-01' - joining_date), dropping the original date column.
- **Complete Case Analysis (CCA):** Drops rows missing the target variable churn_risk_score to prevent target label noise.
- **Hybrid Missing Value Imputation:** Imputes mode for preferred_offer_types, creates an explicit 'Unknown' category for missing region_category, and applies **IterativeImputer (MICE)** (5 iterations, random_state=6) to estimate missing values in numerical features (days_since_last_login, avg_time_spent_in_seconds, avg_frequency_login_days, points_in_wallet).
- **Categorical One-Hot Encoding:** Encodes 12 nominal categorical features using OneHotEncoder(drop='first', sparse_output=False), producing 33 binary features.
- **Multicollinearity Filtering:** Computes pairwise feature correlation matrix and drops features exceeding correlation threshold multicoll_corr_threshold = 0.85.
- **Standardization & Label Zero-Indexing:** Scales features using StandardScaler fitted on train data. Maps raw target 1-5 to zero-indexed integers 0-4.

4. Model Benchmarking & Hyperparameter Optimization

Hyperparameter optimization was conducted using OptunaSearchCV with 5-fold Stratified K-Fold cross-validation across 30 search trials for XGBoost, LightGBM, and CatBoost.

Model Architecture	Best CV Macro F1	Test Accuracy	Test Macro F1	Test Weighted F1	Optimal Hyperparameters (Optuna)
XGBoost Classifier (Champion Model)	0.7725	78.27%	0.7689	0.7777	n_est=107, lr=0.0358, max_depth=8, subsample=0.88, colsample=0.73, gamma=4.04, reg_alpha=0.10, reg_lambda=1.63
LightGBM Classifier	0.7727	77.99%	0.7585	0.7746	n_est=129, lr=0.0392, num_leaves=255, max_depth=8, min_child=27, subsample=0.69, colsample=0.74, reg_alpha=0.37
CatBoost Classifier	0.7700	78.27%	0.7642	0.7725	iterations=144, lr=0.1979, depth=4, l2_leaf_reg=1.42, random_strength=3.68, bagging_temp=0.25

Champion Model (XGBoost) Per-Class Performance Breakdown:

Class Index	Segment Description	Precision	Recall	F1-Score	Test Support
Class 0	Happy / Premium	0.7108	0.8271	0.7646	214
Class 1	Happy / High Value	0.8000	0.6758	0.7327	219
Class 2	Middling	0.9803	0.8357	0.9023	834
Class 3	Unhappy / Drifting	0.7615	0.5471	0.6368	817
Class 4	High Risk Churner	0.6828	0.9898	0.8081	783
Macro Avg	Unweighted Mean	0.7871	0.7751	0.7689	2867
Weighted Avg	Support Weighted	0.8028	0.7827	0.7777	2867

5. In-Depth Inferences & Experimental Insights (Project Notes Audit)

A standout aspect of this project is the rigorous methodological justification documented across three technical notes. Below is a synthesized analysis of each note's findings:

5.1 Note 1 Inference: Tuning Objective Selection & Diagnostic Hierarchy

Note 1 addresses the selection of the scalar optimization target for Optuna hyperparameter search. The core business challenge is navigating opposing operational forces:

- **Catching Churners (Recall):** Requires high recall on churn classes (Class 4 & 3) so that departing customers are identified prior to cancellation.

- **Protecting Marketing Budget (Precision):** Requires high precision to avoid wasting retention discounts on loyal customers (Class 0 & 1) or falsely misclassifying happy users.
- **Why Macro F1 is Superior:** Optuna requires a single scalar target. Macro-averaged F1 balances precision and recall equally per class. Unlike Weighted F1, it prevents majority classes from swamping minority churn classes. Unlike Recall alone, it penalizes precision loss.
- **Post-Tuning Metric Layering:** After tuning with Macro F1, the project computes Class-Specific Recall (missing churners check), Class-Specific Precision (budget efficiency check), Confusion Matrix (misclassification diagnosis between adjacent classes), and Cohen's Kappa (baseline score vs random chance).

5.2 Note 2 Inference: Empirical Confirmation - Macro F1 vs. Recall Tuning

Note 2 documents an empirical experiment directly comparing **Model 1 (Macro F1-tuned)** against **Model 2 (Recall-tuned)**. The empirical results yielded crucial insights into search dynamics:

EMPIRICAL PROOF & TUNING DYNAMICS:

- **Dual Win for Macro F1:** Model 1 (F1-tuned) beat Model 2 (Recall-tuned) on **BOTH Macro F1 (0.769 vs 0.764) AND Macro Recall (0.775 vs 0.771)**. Maximizing recall directly failed to beat the F1 model even on recall itself.
- **Shallow Search Optima:** Recall on Class 4 saturated at ~0.99 in both models (precision ~0.68). Because a recall-only objective provides no penalty for false positives, Optuna's trial sampler wandered into unhelpful search space.
- **Degradation of Hard Classes:** Class 3 recall (the hardest customer segment to predict) *dropped* under the recall-tuned model from **0.547 down to 0.523**. Optimizing recall in isolation actively damaged performance on hard minority classes.
- **Actionable Rule:** Standardizing on Macro-averaged F1 is empirically validated as the superior optimization objective across all future training runs.

5.3 Note 3 Inference: Production Cloud Architecture & Ephemeral IP Decoupling

Note 3 details the production infrastructure built on AWS ECS Fargate, Docker, and Application Load Balancing. Key architectural principles include:

- **Containerized Microservice:** FastAPI app running Uvicorn on 0.0.0.0:8000 inside a Docker image managed with Python 3.12 and uv.
- **Serverless Fargate Execution:** ECS service maintains 1 desired task allocated 0.5 vCPU and 1 GB RAM using awsvpc networking.
- **Application Load Balancer (ALB):** Internet-facing ALB listens on HTTP port 80 and forwards requests to the Target Group IP targets on port 8000.
- **Decoupled Task IP Rationale:** Fargate task public IP addresses are ephemeral and change whenever containers are restarted or replaced during deployments. Routing through the stable ALB DNS name decouples clients from task lifecycle state and enables seamless rolling updates.
- **Active Liveness Health Monitoring:** ALB Target Group performs periodic HTTP health checks against GET / expecting {"status":"ok"}. ECS automatically drains and replaces uninitialized or unhealthy tasks.

6. CI/CD Pipeline & MLOps Automation

The repository features an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline defined in .github/workflows/ci.yml. This workflow automates container building and image publishing to Docker Hub:

- **Trigger Condition:** Executes automatically on every code push to the main branch.
- **Runner Environment:** Runs inside an isolated ubuntu-latest virtual environment.
- **Repository Checkout:** Uses actions/checkout@v4 to fetch source code and repository artifacts.
- **Docker Buildx Setup:** Utilizes docker/setup-buildx-action@v3 to initialize Docker Buildx for optimized multi-platform container builds.
- **Registry Authentication:** Authenticates securely to Docker Hub via docker/login-action@v3 using GitHub Repository Secrets (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN).
- **Automated Build & Push:** Executes docker/build-push-action@v5 targeting ./dockerfile, tags the container image as csagnik1302/ml_img:latest, and publishes the image to Docker Hub.

- **AWS Integration Link:** AWS ECS Fargate tasks pull csagnik1302/ml_img:latest directly from Docker Hub during automated rolling updates, providing a seamless push-to-deploy path.

7. MLOps System Architecture & Serving Flow

The end-to-end operational architecture cleanly decouples offline pipeline execution from online REST/UI serving and automated deployment:

OFFLINE PIPELINE & CI/CD BUILD	ONLINE SERVING & AWS FARGATE
1. Data Ingestion (data/raw/train.csv) 2. Preprocessing & MICE Imputation (src/data/preprocess.py) 3. Optuna Hyperparameter Tuning (5-Fold Stratified CV) 4. Model Training & MLflow Logging (src/models/train.py) 5. GitHub Push to Main (Triggers ci.yml) 6. Docker Hub Push (csagnik1302/ml_img:latest)	1. Client Request (HTTP POST /predict or Gradio /ui) 2. Internet-Facing ALB (Port 80 Public Ingress) 3. ALB Target Group (Health Check GET / on Port 8000) 4. ECS Fargate Task (Pulls latest image from Docker Hub) 5. FastAPI / Uvicorn Server (Pydantic Input Validation) 6. Inference Engine (Loads OHE, Scaler & MLflow Model)

8. Future Roadmap & Enterprise Cloud Enhancements

To elevate this project from a portfolio implementation to an enterprise-grade production platform, the following upgrades are recommended:

- **HTTPS & Transport Security:** Configure SSL/TLS certificates via AWS Certificate Manager (ACM) on ALB port 443 and place Fargate tasks inside private subnets with NAT Gateways.
- **Container Registry Migration:** Migrate container storage from public Docker Hub to Amazon Elastic Container Registry (ECR) for enhanced IAM security and faster internal AWS pulls.
- **Auto-Scaling Capacity:** Establish ECS Target Tracking Scaling policies based on vCPU utilization, memory consumption, or request volume per target.
- **Model Drift Monitoring:** Deploy Evidently AI or Amazon CloudWatch Evidently to monitor real-time feature drift and prediction distribution shifts.